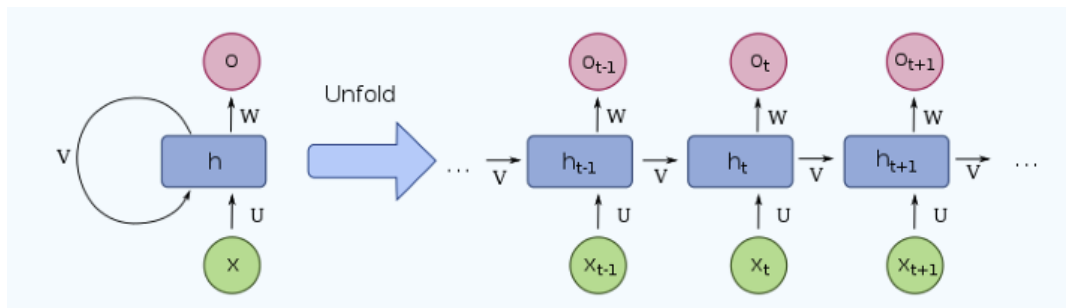


# 5주차 세션

## [ Attention 정리 ]

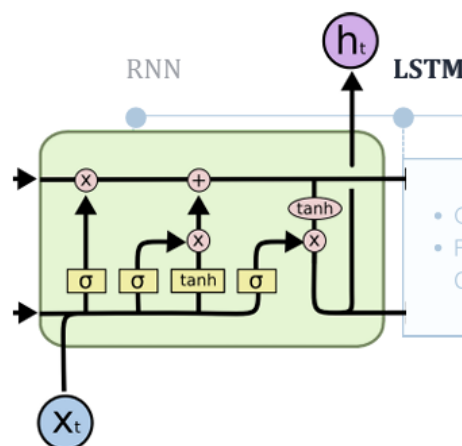
### 1. Before Transformer..

- RNN (Recurrent Neural Network)



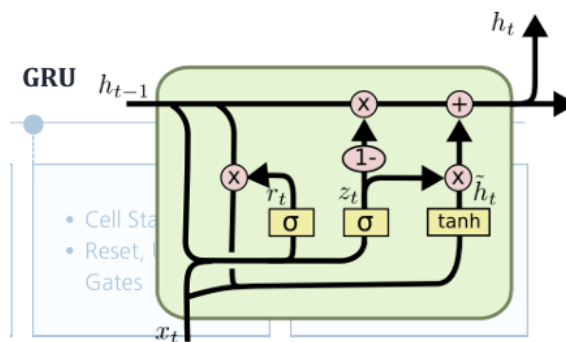
- 시퀀스 데이터를 처리하기 위해 순차적 연산을 수행하는 모델
- 이전 시점의 hidden state를 현재 시점으로 전달하여 문맥을 반영할 수 있음
- 한계 : 긴 문장에서 Gradient Vanishing(기울기 소실) / Explosion(기울기 폭발) 문제 발생 → 장기 의존성 문제 발생
  - 기울기 소실 : 역전파 과정에서 Gradient가 계속 곱해질 때 아주 작은 값이 되어 사라지는 현상

- LSTM (Long Short-Term Memory)



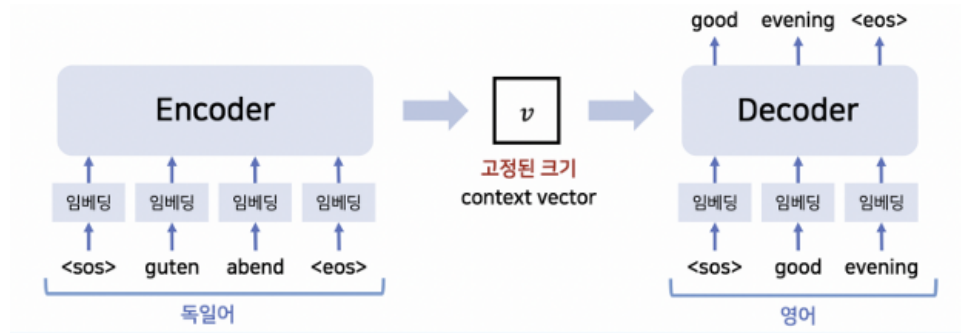
- 장기 의존성 문제를 해결하기 위해 등장
- 핵심은 **Cell State**라는 직접 경로와 세 가지 게이트
  - Forget Gate: 과거 정보를 버릴지 결정
  - Input Gate: 새로운 정보 반영 여부 결정
  - Output Gate: 최종 hidden state 계산
- Forget → Input → Cell State 업데이트 → Output
- 중요한 정보는 유지, 불필요한 정보는 삭제하여 장기 의존성을 학습 가능
- **한계** : 구조가 복잡하고 연산량이 많아 속도가 느림

## • GRU (Gated Recurrent Unit)



- LSTM을 단순화한 모델
- **Reset Gate, Update Gate** 두 개의 게이트만 사용
- Cell State가 따로 없고 hidden state 하나로 관리 → 계산량이 적음
- 성능은 LSTM과 비슷하면서도 학습 속도가 더 빠른 경우가 많음
- Reset → Candidate Hidden State 계산 → Update → Hidden State 업데이트

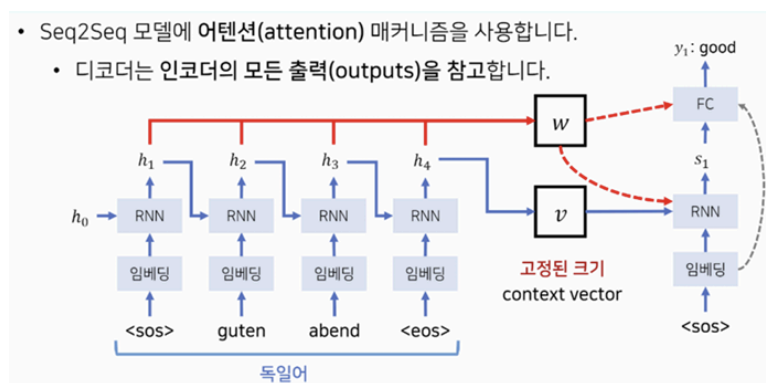
## • Seq2Seq (Sequence-to-Sequence)



- **Encoder-Decoder 구조**로, 입력 시퀀스를 압축된 Context Vector로 변환 후, Decoder가 이를 이용해 출력 시퀀스를 생성
- 일반적으로 RNN 기반으로 구현
- **한계** : 모든 정보를 하나의 벡터에 압축하면서 병목 현상 발생 → 긴 문장에서 성능 저하
  - 병목 현상 : 모든 입력 시퀀스 정보를 하나의 고정된 벡터에 압축하는 과정에서 발생하는 정보 손실 문제

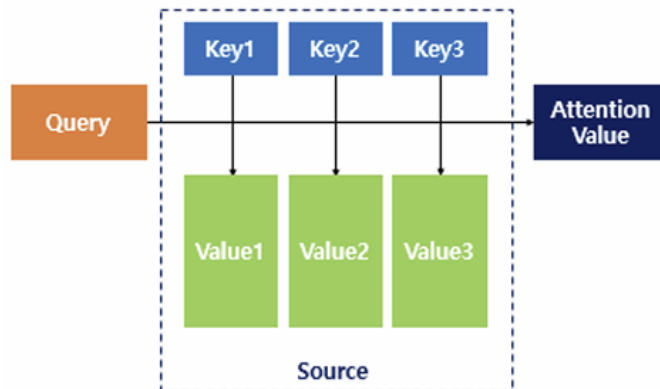
## 2. Attention

- Seq2Seq 모델의 병목 현상과 정보 소실 문제를 해결하기 위함
- 출력 단어를 생성할 때, 예측할 단어와 더 **관련된 입력 단어 부분에 집중(attention)**하자



- 인코더의 모든 출력값들을 별도의 배열에 기록한 후 디코더에서 참고하여 입력 문장 전체를 반영할 수 있도록 함
- 긴 문장에서도 특정 단어와 관련된 모든 단어의 정보를 반영 가능.
- 병목 현상을 줄이고 문맥 이해 능력을 크게 향상.

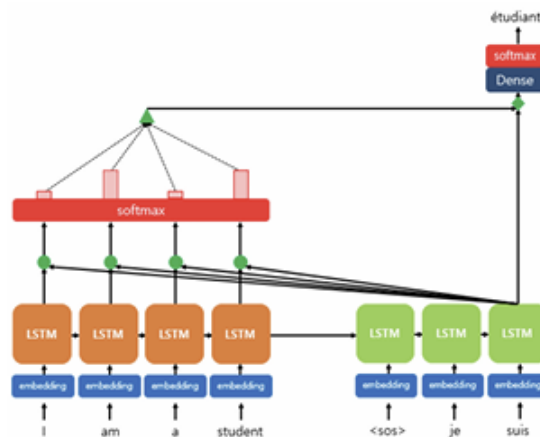
- Q, K, V 구조



- Query (Q): 현재 처리하려는 단어 벡터
- Key (K): Query와 얼마나 비슷한지 알아보기 위해 유사도를 측정 당하는 벡터
- Value (V): Q-K 간에 실제로 구해진 유사도 가중치가 적용되는 값 벡터

- Dot-Product Attention 과정

### “Dot-Product Attention”



1. Query와 Key 내적 → **Attention Score**  
⇒ 디코더 hidden state(=Query)와 인코더 hidden states(=Keys)의 유사도를 계산
2. softmax로 확률화 → **Attention Weight**  
⇒ score들을 softmax로 확률 분포화해서 “어디를 얼마나 집중할지” 결정

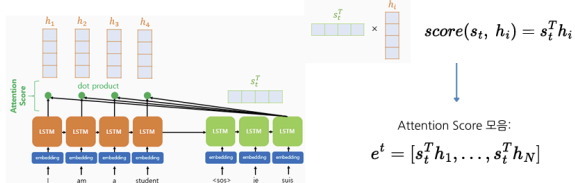
### 3. Attention Weight × Value = **Attention Value**

⇒ 각 인코더 hidden state(=Value)에 가중치를 곱해서 가중합(=Context Vector=Attention)을 구함

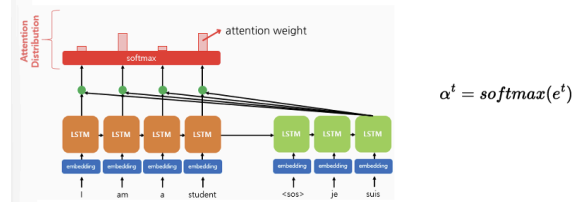
### 4. Attention Value와 Decoder hidden state를 연결 → 최종 출력

⇒ Context Vector와 디코더 hidden state를 합쳐서(Concat + 변환) 최종 출력 단어 예측

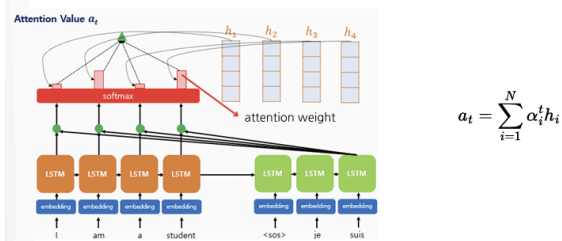
1) Attention Score 구하기



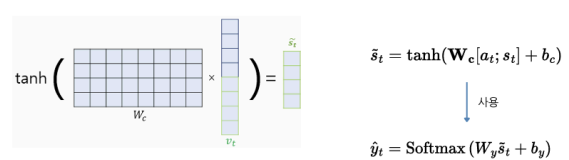
2) softmax 함수를 통해 Attention 분포 구하기



3) 각 인코더의 어텐션 가중치와 은닉 상태를 가중합하여 Attention Value 구하기

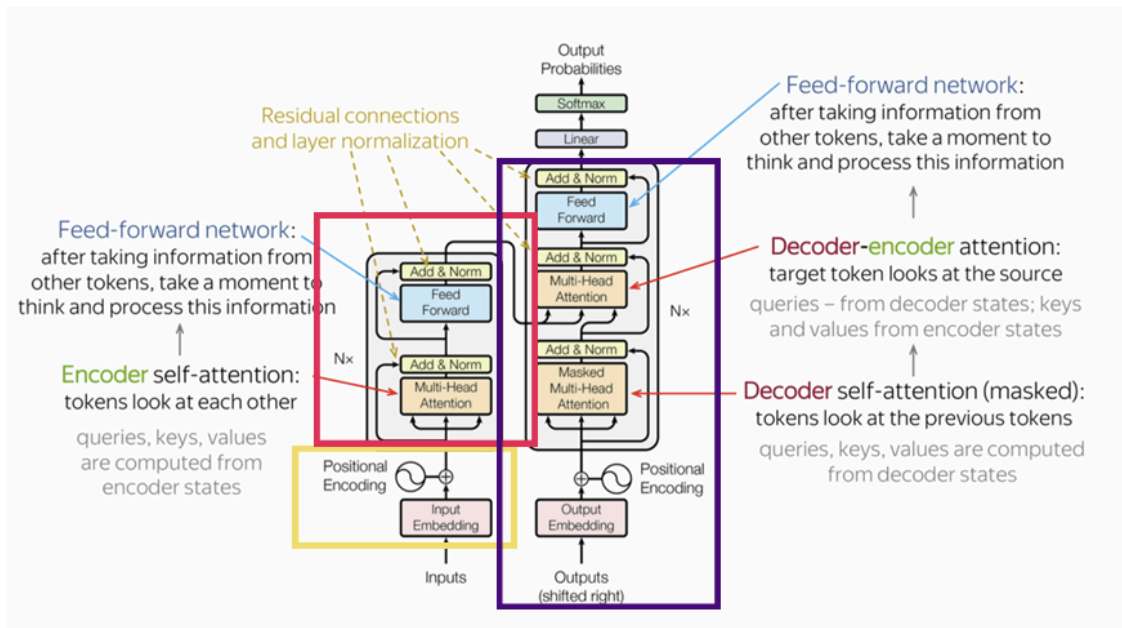


5) 출력층 연산의 입력이 되는  $\tilde{s}_t$  를 계산



★ Attention을 통해 현재 출력 대상에 대해 모든 인코더의 hidden state를 고려할 수 있음!

## 3. Transformer



- RNN 계열을 제거하고 **Attention만으로 구성된 모델**.
  - 기존 Seq2Seq + RNN 기반 모델의 한계점 극복 (느린 학습속도 / 순차처리 / 긴 문장에서 장기 의존성)
  - RNN 없이 병렬 연산 + Self-Attention으로 문맥 학습
- **Encoder-Decoder 구조 유지**
- 입력 전체를 동시에 처리 가능

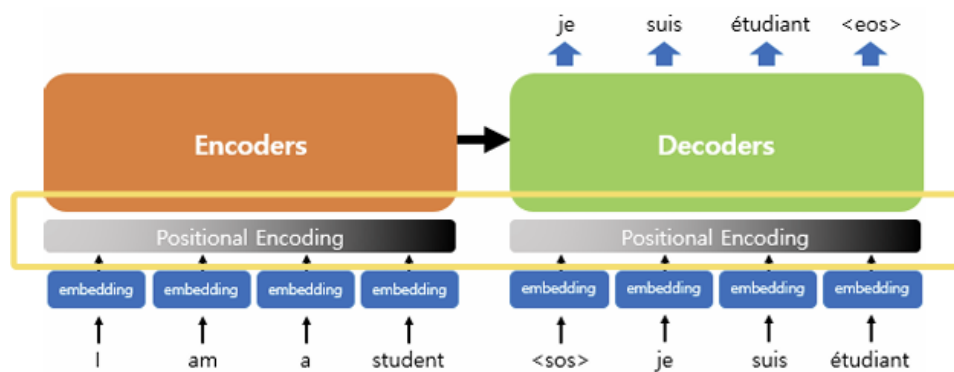
## Input into Encoder

- **Embedding**
  - 각 단어를 학습 가능한 벡터로 변환
  - 임베딩 행렬  $W$ 에서 look-up 수행
    - 입력 문장이 단어 ID로 들어오면 그 ID를 인덱스로 해서  $W$ 에서 해당 행을 뽑는 과정 → 순서정보X



## • Positional Encoding

- Transformer는 순서 정보가 없으므로 위치값 추가
- Sine, Cosine 함수를 이용해 각 토큰 위치를 고유하게 인코딩 (멀수록 큰 값)
- 입력이 아무리 길어져도 위치 정보 반영 가능

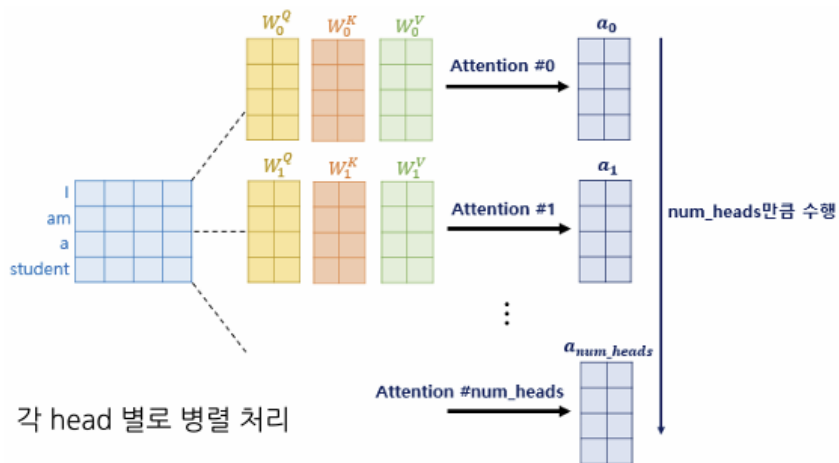


## Encoder

### • Self-Attention

- 문장 안 단어들 사이의 연관성을 학습 → Attention과 원리는 같음
- Query, Key, Value를 모두 같은 입력(문장)에서 생성

### • Multi-Head Attention

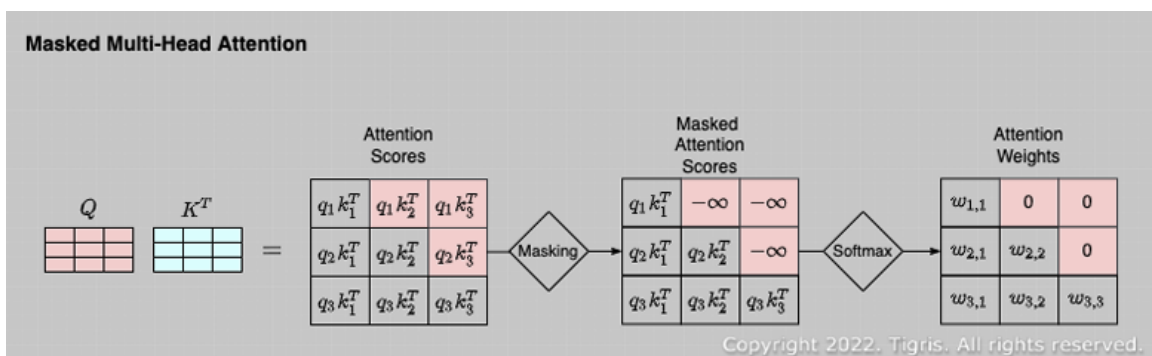


- Self-Attention을 여러 head로 병렬 수행
- 각 head는 서로 다른 의미적 관계를 학습
- Concat 후 선형 변환으로 통합 → 입력 sequence의 vector와 동일한 크기의 결과 출력

## Decoder

### • Masked Multi-Head Attention

- 미래 단어를 보지 못하도록 미래 정보의 attention score를  $-\infty$  처리 후 softmax
  - Transformer는 문장 전체를 한번에 입력받아 미래의 정보를 확인 가능하기 때문
  - 순차적 예측 가능



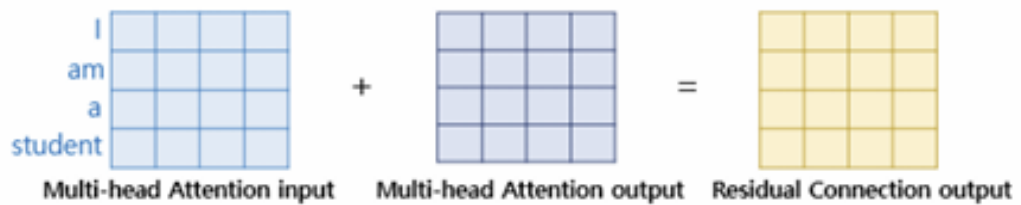
### • Feed-Forward Network

- ReLU와 두 번의 선형 변환을 적용
- 출력 벡터는 입력 전과 동일하게 보존됨

### • Residual Connection + Layer Normalization



- Residual Connection
  - Multi-head Attention의 입력과 출력 더함
  - 기울기 소실 방지, 학습 안정성 확보



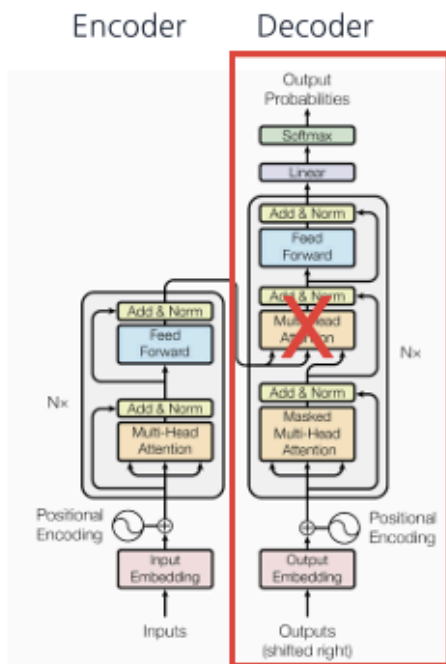
- 이후 Layer Normalization으로 Residual Connection 결과 정규화

## 4. GPT, BERT, BART

| Model          | Structure               | Method                                              | Direction       |
|----------------|-------------------------|-----------------------------------------------------|-----------------|
| <b>GPT</b>     | Decoder                 | Auto-Regressive                                     | Unidirection    |
| <b>BERT</b>    | Encoder                 | Auto-Encoding                                       | Bidirection     |
| <b>BART</b>    | Encoder + Decoder       | Auto-Encoding + Auto-Regressive                     | Bi+Unidirection |
| <b>ELECTRA</b> | Encoder + Discriminator | Auto-Encoding + Replaced Token Detection            | Bidirection     |
| <b>T5</b>      | Encoder + Decoder       | Auto-Encoding + Auto-Regressive + Text-to-Text task | Bidirection     |

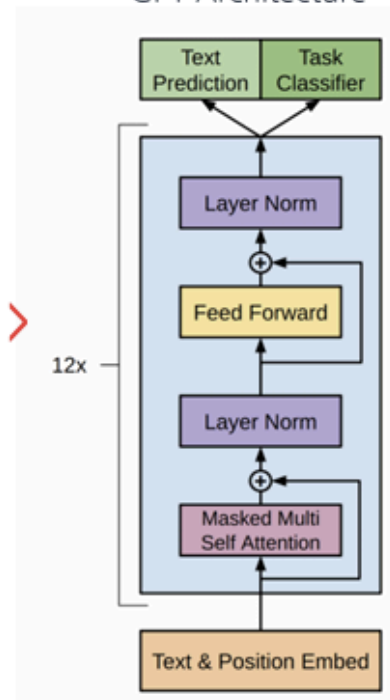
- **GPT**

## Transformer Architecture



Decoder only >

## GPT Architecture



- Transformer의 Decoder 기반
- Auto-Regressive 방식으로 순차적으로 다음 단어 예측
  - Unsupervised Pre-training** → Next Word Prediction (다음 단어 예측)
    - 큰 코퍼스에서 문맥 기반으로 다음 단어를 맞추는 방식
    - 장기 문맥 이해 가능

$$L_1(\mathcal{U}) = \sum \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

$k$  : size of context window

$\Theta$  : neural network parameters

ex) The weather is nice today.  $k=3$

>  $\max \log P(\text{nice} | \text{the, weather, is}) + \log P(\text{today} | \text{weather, is, nice})$

$$h_0 = U \overbrace{W_e}^{\text{: token embedding}} + \overbrace{W_p}^{\text{: position embedding}}$$

$$h_l = \text{transformer\_block}(h_{l-1}) \forall i \in [1, n] \quad n = \# \text{ transformer block}$$

$$P(u) = \text{softmax}(\overbrace{h_n W_e^T}^{\text{: output of final layer}})$$

> 각 위치에 대해 다음 토큰이 나올 확률 분포

- Supervised Fine-tuning** → task-specific 학습
  - 사전학습된 모델을 task 데이터셋에 맞춰 미세조정

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

For labeled dataset  $\mathcal{C}$ ,  $x^1 \dots x^m$  : input token sequence,  $y$  : label

$$P(y | x^1, \dots, x^m) = \text{softmax}(h_l^m W_y) \quad \text{Task Classifier head weight}$$

$$L_2(\mathcal{C}) = \sum_{(x,y)} \log P(y | x^1, \dots, x^m)$$

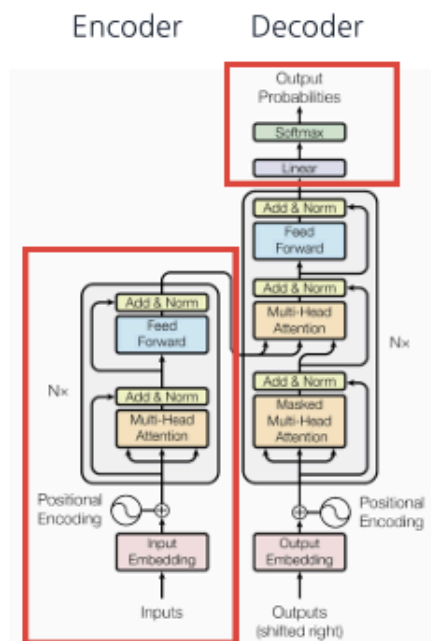
$$L_3(\mathcal{C}) = L_2(\mathcal{C}) + \lambda * L_1(\mathcal{C}) \quad \text{Auxiliary Learning / Objective}$$

hyperparam (set to 0.5)

- GPT-1 (2018) → GPT-2 (Zero-shot) → GPT-3 (Few-shot) → GPT-3.5 (RLHF) → GPT-4 (Multimodal) → GPT-5 (2025)

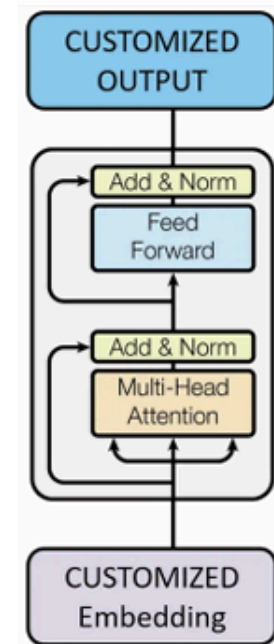
## • BERT

Transformer Architecture



Encoder only >

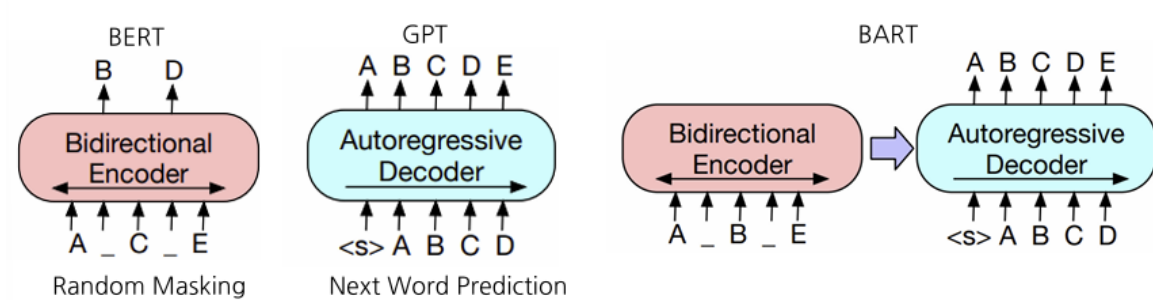
BERT Architecture



- Transformer의 Encoder 기반
- 문장을 양방향으로 동시에 이해 가능
- Masked Language Model (MLM) + Next Sentence Prediction (NSP)
  - **Unsupervised Pre-training → MLM + NSP**
    - 문장에서 일부 토큰을 마스킹하고 복원 (MLM)
    - 두 문장이 실제 연속 문장인지 예측 (NSP)

- **Supervised Fine-tuning → task-specific 학습**
  - 사전학습된 모델을 downstream task에 맞게 미세조정
  - [CLS] 토큰 활용 → 분류기 연결
- QA, 자연어 추론(NLI), 문서 분류 등 랩양한 이해 기반 작업에 활용됨

## • BART



- Bidirectional (BERT) Auto-Regressive (GPT) Transformer
- Denoising Autoencoder 방식
  - 문장 구조의 의미를 보존하면서 다양한 변형을 학습
  - 입력 문장에 다양한 노이즈 (삭제, 치환, 순서 섞기 등)를 추가한 뒤 이를 원래 문장으로 복원하도록 학습